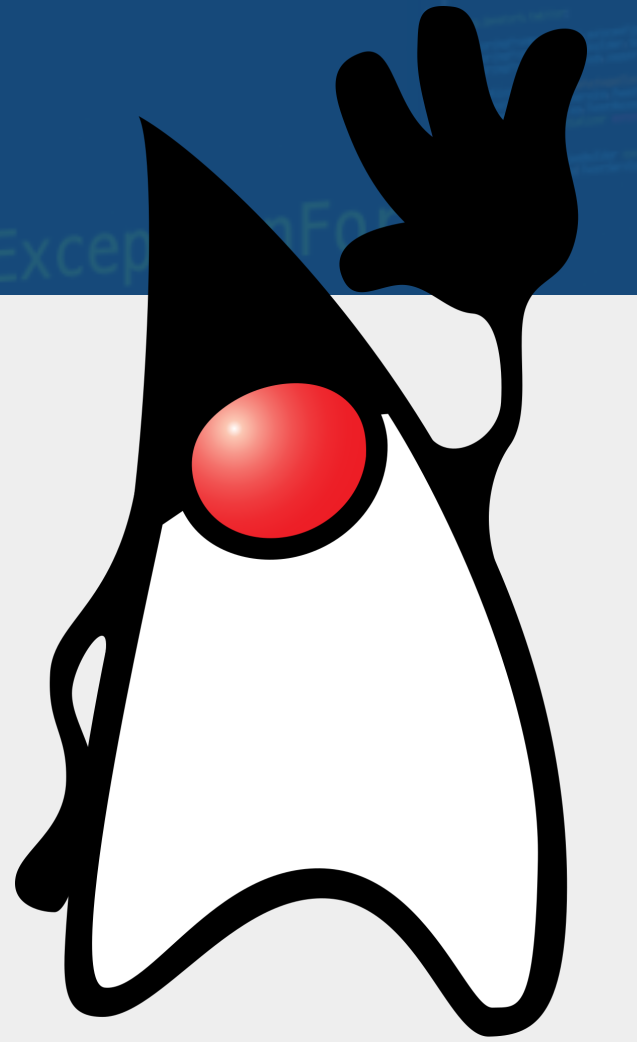


# Java Programlama Dili

## 8. Bölüm

# Değişkenlerin Özellikleri

Akın Kaldıroğlu



# Amaç



- Bu bölümde daha önce giriş yapılan değişkenlerle ilgili temel özellikler ele alınacaktır

# Konular



- **Değişkenler**

- Değişken Tanımlama

- **Değişkenlerin Roller**

- Sınıf ve Nesne Değişkenleri
- Yerel Değişkenler
- var ile Yerel Değişken Tanımlama
- İsimsiz Değişken

- **Bellek**

- Heap

- Stack

- Değişkenlerin Ömrü

- **Kapsam**

- Global Değişken

- **Değişmezlik**

- Sabit İfade ve Sabit Değişken

- **Ödevler**

- **Bundan Sonra?**

# Değişkenler



# Veri - Değişken - I



- Programlama dillerinde, sayı, karakter, nesne vb. türlerden veriler (data) ile bu verileri tutan **değişkenlerin (variable)** olduğunu öğrendik
- Ve şunları da ekledik:
  - Değişkenler, verilerin soyutlanmış sunumlarıdır (abstract representation)
  - Değişkenler, verileri program içinde gezdirmek için kullanılır
  - Veriler de onları tutan değişkenler de bellekte bulunur, orada yaşarlar
    - Bellek konusunu birazdan detaylandıracağız

# Veri - Değişken - II

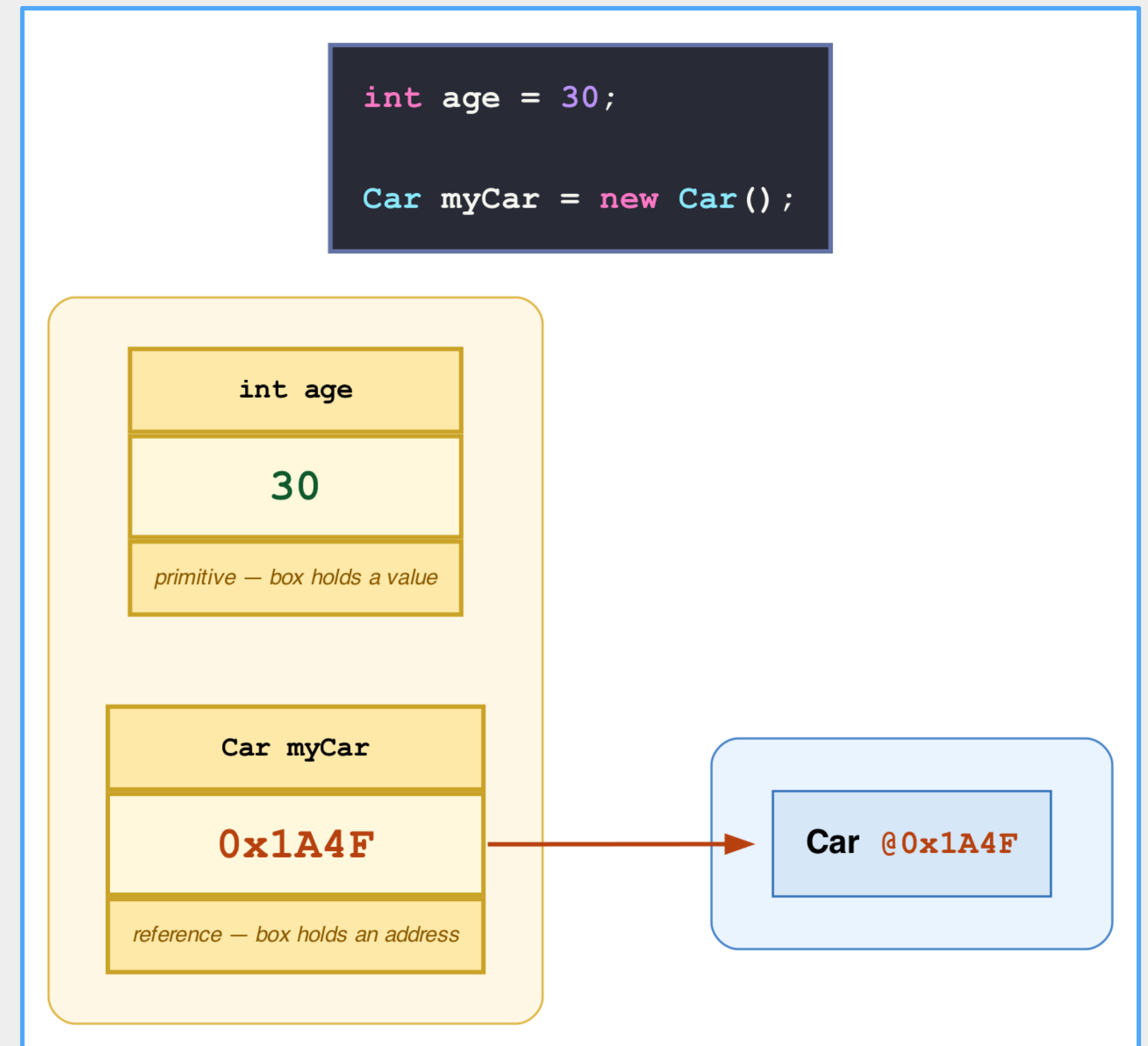


- Java'da veriler, yapı olarak ilkel (primitive) ya da karmaşık (complex ya da referans/reference veya nesne/object) olabilir de dedik
- Java'da değişkenlerin de tipi vardır ve bu tip değişkenin tuttuğu/tutacağı ya da gösterdiği/göstereceği verinin tipi tarafından belirlenir
- Dolayısıyla Java'da değişkenler, ilkel ya da karmaşık yani referans tiplerden olur

# Veri - Değişken - III



- Kutu benzetmesi ile ilkel değişkenlerin verinin değerini saklarken referans değişkenlerin gösterdiği nesnenin adresini sakladığını da öğrendik



# Veri - Değişken - IV



- Öte yandan karmaşık tiplerin birer nesne olduklarını ve veriden daha fazlasına sahip olduklarını da gördük
- Karmaşık tipler
  - birden fazla veriyi bir araya getirir,
  - ve metotlar yoluyla davranışlara da sahiptir



# Değişken Tanımlama

# Değişkenlerin Değeri



- Java'da değişkenler, değerleri kullanılmadan önce tanımlanmalıdır

```
int counter;  
int i = counter; // Error!  
System.out.println(counter); // Error!
```

- Çünkü Java'da değişkenler tanıtılırken onlara bir varsayılan değer atanmaz
  - İstisnaları tabi ki var!
- Değeri olmayan değişkenler de kullanılamaz, derleyici hata verir

# Değişken Tanımlama (Definition)



- Değişken tanımlamak (definition), iki adımda, değişkeni tanıtmak (declaration) ve ona bir ilk değer atamakla (initialization) gerçekleşir

Değişken Tanımlama = Değişken Tanıtma + İlk Değer Atama

Variable Definition = Variable Declaration + Initialization

```
int counter = 7;  
boolean closed = true;  
Student student = new Student();
```

# Değişken Tanıtma (Declaration)



- Değişkeni tanıtmak, değişkenin tipi ve ismiyle bir tanıtma cümlesi kurmak (declaration statement) ve bu sayede değişkeni derleyiciye bildirmek demektir
- Unutmayın Java güçlü tipli bir dildir ve her değişkenin bir tipi vardır ve bu tip tanımlandıktan sonra asla değişmez
- Değişken *tanıtımında* ilk değer ataması yapılmaz; ilk değer ataması yapılırsa değişken *tanımlanmış* olur

```
tip isim;
```

```
int counter;  
boolean closed;  
Student student;
```

# İlk Değer Atama (Initialization) - I



- Derleyiciye ismi ve tipiyle tanıtılmış (declared) olan bir değişkene, tipiyle uyumlu bir ilk değer ataması yapılmalıdır,
- Aksi taktirde değeri olmadığından, değişken kullanılamaz
- Tanımlanmamış bir değişkenin değerine ulaşmaya çalışmak bir derleme hatasıdır
- Ancak ilk değer atamasından sonra değişken *tanımlanmış (defined)* olur

```
int i;  
int j = i; // Error!  
System.out.println(i); // Error!
```

```
int i = 5;  
int j = i; // OK!  
System.out.println(i); // OK!
```

```
int i;  
i = 5;  
int j = i; // OK!  
System.out.println(i); // OK!
```

# İlk Değer Atama (Initialization) - II



- Değişken oluşturulurken ilk değeri bilinmiyorsa, sadece tanıtılır, ilk değer ataması sonraya ertelenir

```
int counter;  
...  
counter = 5;
```

```
char character;  
...  
character = 'c';
```

```
Student student;  
...  
student = new Student();
```

# İlk Değer Atama (Initialization) - III



- Tanıtılırken ilk değer biliniyorsa, atama yapılabilir
- Çoğu zaman ilk değer, kullanıcı tarafından girilerek, hesaplanarak ya da başka bir sistemden getirilerek çalışma zamanında atanması beklenir

```
int counter = 5;
```

```
int counter;  
...  
counter = 5;
```

```
int counter = 5;
```

```
int counter;  
...  
int x = 5;  
...  
counter = x;
```

```
Student student = new Student();
```

```
Student student;  
...  
student = new Student();
```

# Not



- Bir cümlede birden fazla aynı tipten değişken tanımlanabilir ve tanımlanabilir
- Ama bir cümlede birden fazla değişkene değer ataması yapılamaz

```
int i, j, k; // Many declarations of the same type
int i = 1, j = 2, k = 3; // Many definitions of the same type

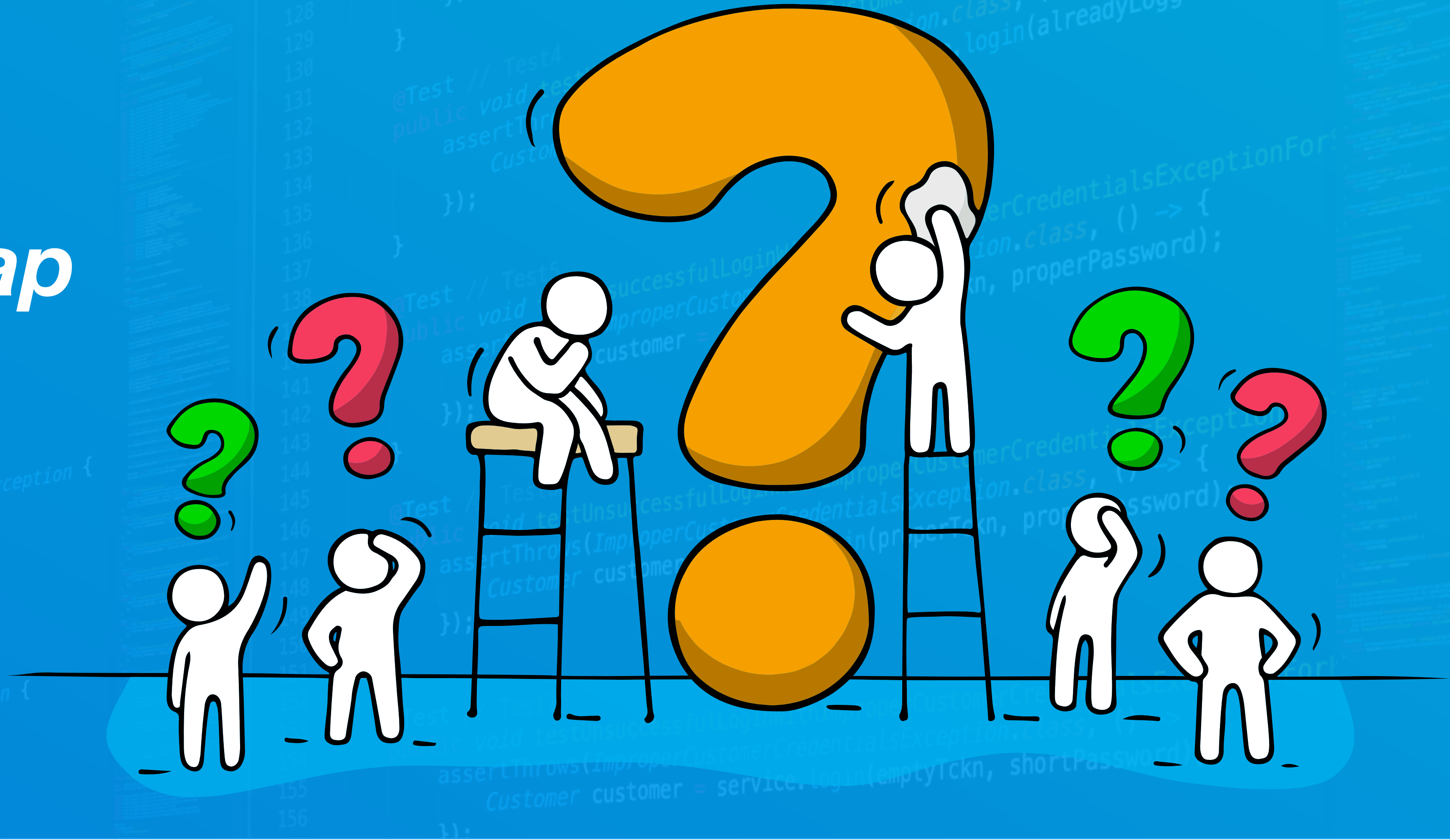
int i, j, k;
i = 1, j = 2, k = 3; // Many initializations is error!
```

# Variables



- **Variables**
- Bu örnekte, değişken tanıma, ilk değer atama ve tanımlama örnekleri vardır
- Tanımlanmamış değişkenlerin değerlerine erişmeye çalışınca Java derleyicisinin hata verdiğini gözlemleyin
- Bu hatayı gidermek için ilkel tiplere uygun ilk değer ile referans tiplere en azından `null` ilk değeri verilmesi gerektiğini gözlemleyin

# Soru ve Cevap Zamanı!



# Değişkenlerin Rolleri



# Değişkenlerin Roller - I



- Java'da değişkenler, ilkel ya da referans, programdaki fonksiyonları ya da rolleri açısından üçe ayrılır:
- **Nesne değişkenleri (instance ya da object variables):** Nesnenin durumunu (state) oluşturan değişkenlerdir, nesneyi tasvir ederler
- **Sınıf değişkenleri (class variables):** Nesnelerin ortak durumunu ifade eden değişkenlerdir, nesneyi tasvir ederler
- **Yerel değişkenler (local variables):** Yerel ya da geçici değişkenlerdir, nesneyi tasvir etmezler

# Değişkenlerin Roller - II



- Biz önce kısaca nesne ve sınıf değişkenlerini sonra yerel değişkenleri inceleyeceğiz
- Sonrasında kapsam ve değişmezlik gibi değişkenlerin genel özelliklerini ele alacağız
- Nesne ve sınıf değişkenlerini ilerideki bölümlerde daha detaylıca ele alacağız



# Nesne ve Sınıf Değişkenleri

# Nesne ve Sınıf Değişkenleri - I



- **Nesne değişkenleri** (**instance** ya da **object variables**), nesnenin durumunu (the state of the object) oluşturan değişkenlerdir
- **Nesnenin durumu** (**the state of the object**), nesnenin o andaki tüm özelliklerinin toplamıdır
- Nesne değişkenleri, nesnenin parçasıdır ve nesneyi betimlemekte kullanılır
- Sınıf değişkenleri ise o sınıftan oluşturulan nesnelerin ortak değere sahip durumlarını ifade eder

# Nesne ve Sınıf Değişkenleri - II



- Nesne ve sınıf değişkenlerine **üye değişken (member variable)** ya da **alan (field)** denir
- Nesne ve sınıf değişkenleri, nesne merkezli programlamada en önemli değişkenlerdir
- Nesne değişkenlerine, nesnenin referansı üzerinden ulaşılır
- Sınıf değişkenlerine, hem sınıf ile hem de nesnelerinin referansları üzerinden ulaşılır

# Nesne ve Sınıf Değişkenleri - II



- Üye değişkenler yani nesne ve sınıf değişkenleri, sınıfın içinde (sınıf bloğunda) ama herhangi bir alt bloğun dışında tanıılır
- Java 25 ile bunun istisnasının olduğunu biliyoruz
- En temel alt blok, metot bloğudur
- Üye değişkenler sınıf bloğunda ve genelde metotlardan önce tanıılır ve sıklıkla ilk değerlerini metotlarda alır

```
<niteleyici>* class <Sınıfİsmi>{  
    <nesne değişkeni>*  
    <sınıf değişkeni>*  
    <metot>*  
}
```

# Not: BNF - Backus-Naur Form



- **BNF (Backus-Naur Form)**, programlama dillerinin söz dizimini (syntax) tanımlamak için kullanılan bir gösterimdir
- Açılı parantezler `< . . . >` bir söz dizimsel öğeyi (yer tutucuyu) gösterir, `*` ise *sıfır veya daha fazla kez tekrarlanabilir* anlamını taşır
- Yani `<nesne değişkeni>*` ifadesi, *sınıfın içinde sıfır veya daha fazla nesne değişkeni bulunabilir* demektir, buradaki `*` bir çarpma işlemi değildir

```
<niteleyici>* class <Sınıfİsmi>{  
    <nesne değişkeni>*  
    <sınıf değişkeni>*  
    <metot>*  
}
```

# Car.java



- **Car** sınıfında 5 tane nesne değişkeni, 4 tane metot vardır
- Hiç sınıf değişkeni yoktur
- Nesne değişkenleri, sınıf bloğunda ve sınıfın başında tanıtılmıştır

```
class Car{

    String make;
    String model;
    String year;
    int speed;
    int distance;

    double go(int newDistance) {
        ...
    }

    void accelerate(int newSpeed) {
        ...
    }

    void stop() {
        ...
    }

    String getInfo() {
        ...
    }
}
```

# CarTest.java - I



- `CarTest` sınıfının `main` metodunda önce `Car` sınıfının bir nesnesi oluşturulmuş ve `myCar` referansına atanmıştır
- Sonrasında `Car` nesnesinin durumu belirlenmiştir
- `Car` nesnesinin durumu, nesne değişkenlerinin değerlerinin bütünüdür

```
class CarTest{
```

```
    public static void main(String[] args) {
```

```
        Car myCar = new Car();
```

```
        myCar.make = "Toyota";
```

```
        myCar.model = "Corolla";
```

```
        myCar.year = "2024";
```

```
        myCar.speed = 80;
```

```
        myCar.distance = 1500;
```

```
        ...
```

```
    }
```

```
}
```

Nesnenin durumu  
(state of object)

# CarTest.java - II



```
class CarTest{
```

```
    public static void main(String[] args) {
```

```
        Car myCar = new Car();
```

```
        myCar.make = "Toyota";
```

```
        myCar.model = "Corolla";
```

```
        myCar.year = "2024";
```

```
        myCar.speed = 80;
```

```
        myCar.distance = 1500;
```

```
        ...
```

```
    }  
}
```

Nesnenin durumu  
(state of object)

```
Car myCar = new Car();
```

```
myCar.make = "Toyota";
```

```
myCar.model = "Corolla";
```

```
myCar.year = "2024";
```

```
myCar.speed = 80;
```

```
myCar.distance = 1500;
```

Stack

Car myCar

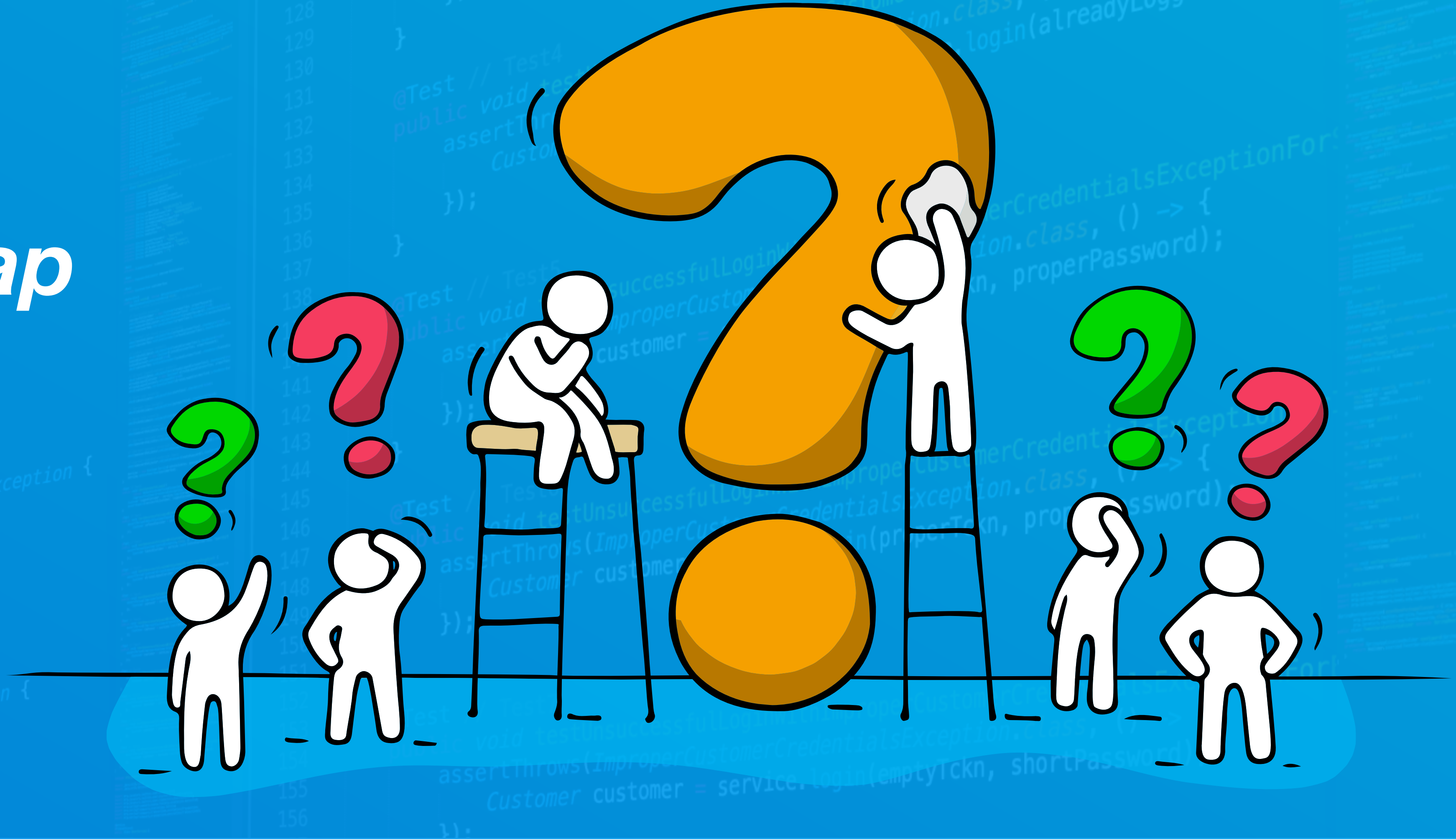
0x1A4F

Heap — all member variables live inside the object

Car @0x1A4F

|          |           |
|----------|-----------|
| make     | "Toyota"  |
| model    | "Corolla" |
| year     | "2024"    |
| speed    | 80        |
| distance | 1500      |

# Soru ve Cevap Zamanı!





# Yerel Değişkenler

# Yerel Değişkenler - I



- Sınıfın alt bloklarının içinde tanıtilan dolayısıyla da tanımlanan değişkenlere **yerel değişken (local variable)** denir
- Yerel değişkenler, sınıfın ya da nesnenin değil, içinde tanıtıldıkları dolayısıyla da tanımlandıkları bloğun parçasıdır, bu yüzden onlara **yerel (local)** denir
- Metot parametreleri ve metotlarda ve metotların içindeki alt bloklarda tanıtilan/tanımlanan değişkenler, yerel değişkendir

# Yerel Değişkenler - II



- Yerel değişkenler, nesnenin parçası değildir,
  - Yani yerel değişkenler nesnenin durumunun parçası değildir, dolayısıyla nesnenin referansı üzerinden ulaşılamaz!
- Yerel değişkenler, nesnelerin gösterdikleri davranışlarda geçici değerleri tutan değişkenlerdir
  - *Kullan at* şeklinde çalışırlar
  - Ama nesne değişkenleri bellekte durmaya devam ederler

# Car.java - I



- **Car** sınıfında 5 tane nesne değişkeni, 4 tane metot vardır
- Hiç sınıf değişkeni yoktur
- Nesne değişkenleri, sınıf bloğunda ve sınıfın başında tanıtılmıştır

```
class Car{

    String make;
    String model;
    String year;
    int speed;
    int distance;

    double go(int newDistance) {
        ...
    }

    void accelerate(int newSpeed) {
        ...
    }

    void stop() {
        ...
    }

    String getInfo() {
        ...
    }
}
```

# Car.java - II



- **Car** sınıfının metotlarında tanıtilan yerel değişkenlerin altı çizilmiştir
- Metotlarda kullanılan **make**, **model**, **year**, **distance**, **speed** ise nesne değişkenidir

```
//Car sınıfındaki yerel değişkenlerin altı çizilmiştir.
class Car{

    ...
    double go(int newDistance) {
        distance += newDistance;
        double period = (double) newDistance / speed;
        return period;
    }

    void accelerate(int newSpeed) {
        speed = newSpeed;
    }

    void stop() {
        speed = 0;
    }

    String getInfo() {
        String info = "Car Info: " + year + " " + make + " " +
            model + ". Distance: " + distance + " km. and
            traveling at " + speed + " kmph.";

        return info;
    }
}
```

# CarTest.java



- `CarTest` sınıfının `main` metodu çalıştırılınca aşağıdaki gibi bir bellek yapısı oluşur

```
public class CarTest{  
  
    public static void main(String[] args) {  
        Car myCar = new Car();  
        myCar.make = "Toyota";  
        myCar.model = "Corolla";  
        myCar.year = "2024";  
        myCar.distance = 1500;  
        myCar.speed = 80;  
  
        myCar.go(200);  
    }  
}
```

When `myCar.go(200)` is called: state of `go()`'s stack frame and the `Car` object:

```
public double go(int newDistance) {  
  
    distance += newDistance;  
  
    double period = (double) newDistance / speed;  
  
    return period;  
}
```

Stack — `go()` frame: local variables

|             |     |
|-------------|-----|
| newDistance | 200 |
| period      | 2.5 |

Heap — `Car` object: member variables

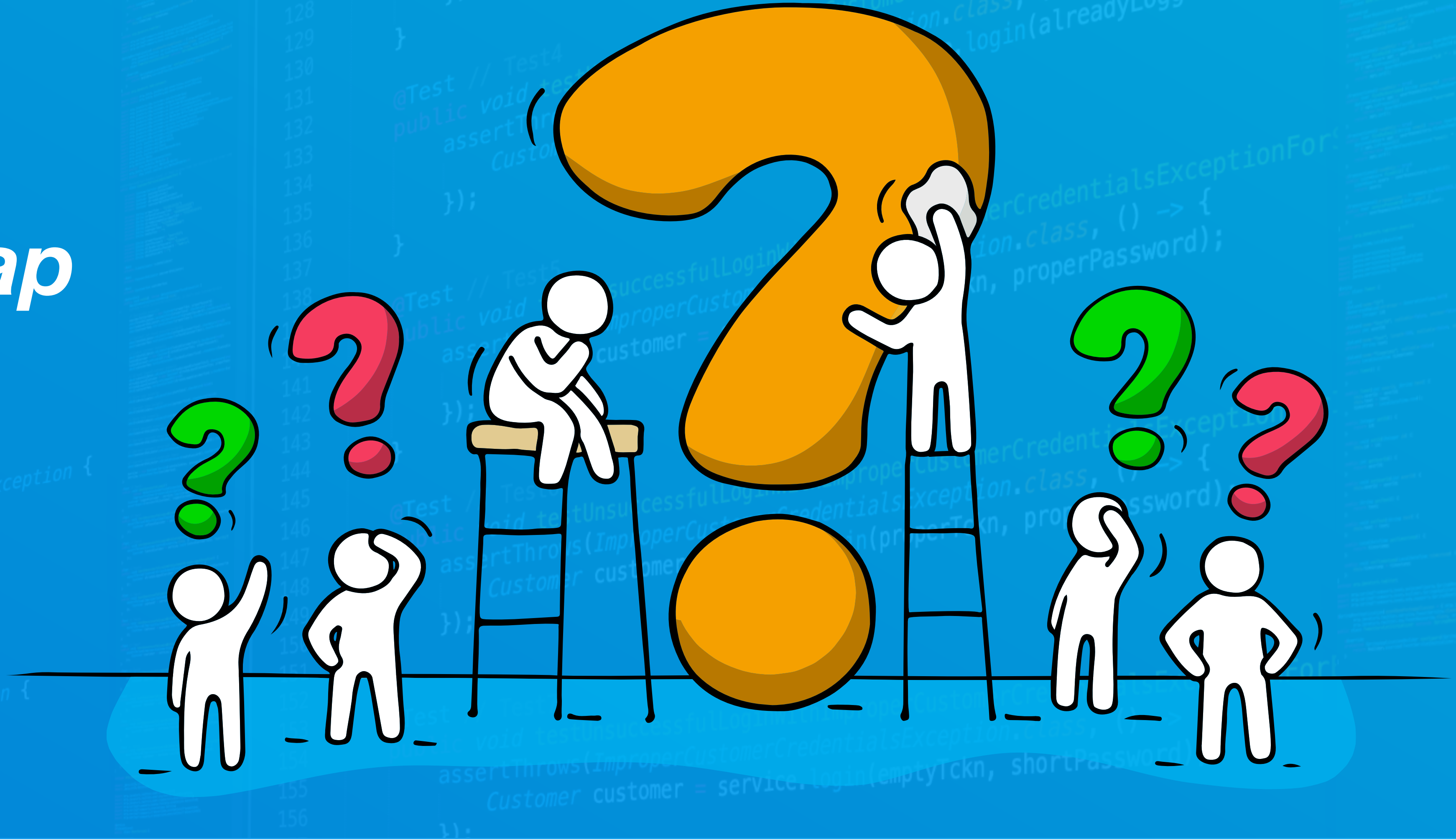
| Car @0x1A4F           |           |
|-----------------------|-----------|
| make                  | "Toyota"  |
| model                 | "Corolla" |
| year                  | "2024"    |
| speed                 | 80        |
| distance              | 1700      |
| after distance += 200 |           |

# Car ve CarTest



- `Car` ve `CarTest`
- Bu örnekte `Car` sınıfındaki nesne ve yerel değişkenleri incelenecektir

# Soru ve Cevap Zamanı!





# var ile Yerel Değişken Tanımlama

# var - I



- Java 10 ile birlikte yerel değişkenlerin **var** kullanılarak tanımlanması mümkün hale geldi
- Eğer bir yerel değişken **var** ile tanımlanırsa, tipinin belirtilmesine gerek yoktur

```
var i = 5; // i is of type int  
var c = new Car(); // c is of type Car
```

- Bu durumda Java derleyicisi tip bilgisini bağlamdan (context) çıkarır
- Buna **tip çıkarımı (type inference)** denir

# var - II



- **var** ile değişken tanıtilamaz sadece tanımlanabilir
- Değişkene bir ilk değer verilmezse derleyici tip çıkarımı yapamaz dolayısıyla hata oluşur
- **var** ile hem ilkel hem de karmaşık tipte değişken oluşturulabilir

```
var i = 5; // i, int tipinde
int i = 5; // Yukarıdaki satır ile aynı
var i;     // Hata! Tip bilgisi çıkarılamaz!

var d = 3.0; // d, double tipinde
double d = 3.0; // Yukarıdaki satır ile aynı
var b = false;

var c = new Car();
Car c = new Car();
```

# var - III



- **var** ile sadece yerel değişken oluşturulabilir, üye değişken oluşturulamaz

```
class Car{  
    var make = "Xxx"; // Compiler error!  
  
    var model;        // Compiler error!  
    ...  
}
```

# var - IV



- **var**, anahtar kelime (keyword) değildir, ayrılmış kelimedir (reserved word)
- Bu sebeple Java 10 ile derlenecek olan, **var**'ı değişken, metot ve paket ismi olarak kullanan kodlarda bir değişiklik yapmaya gerek yoktur

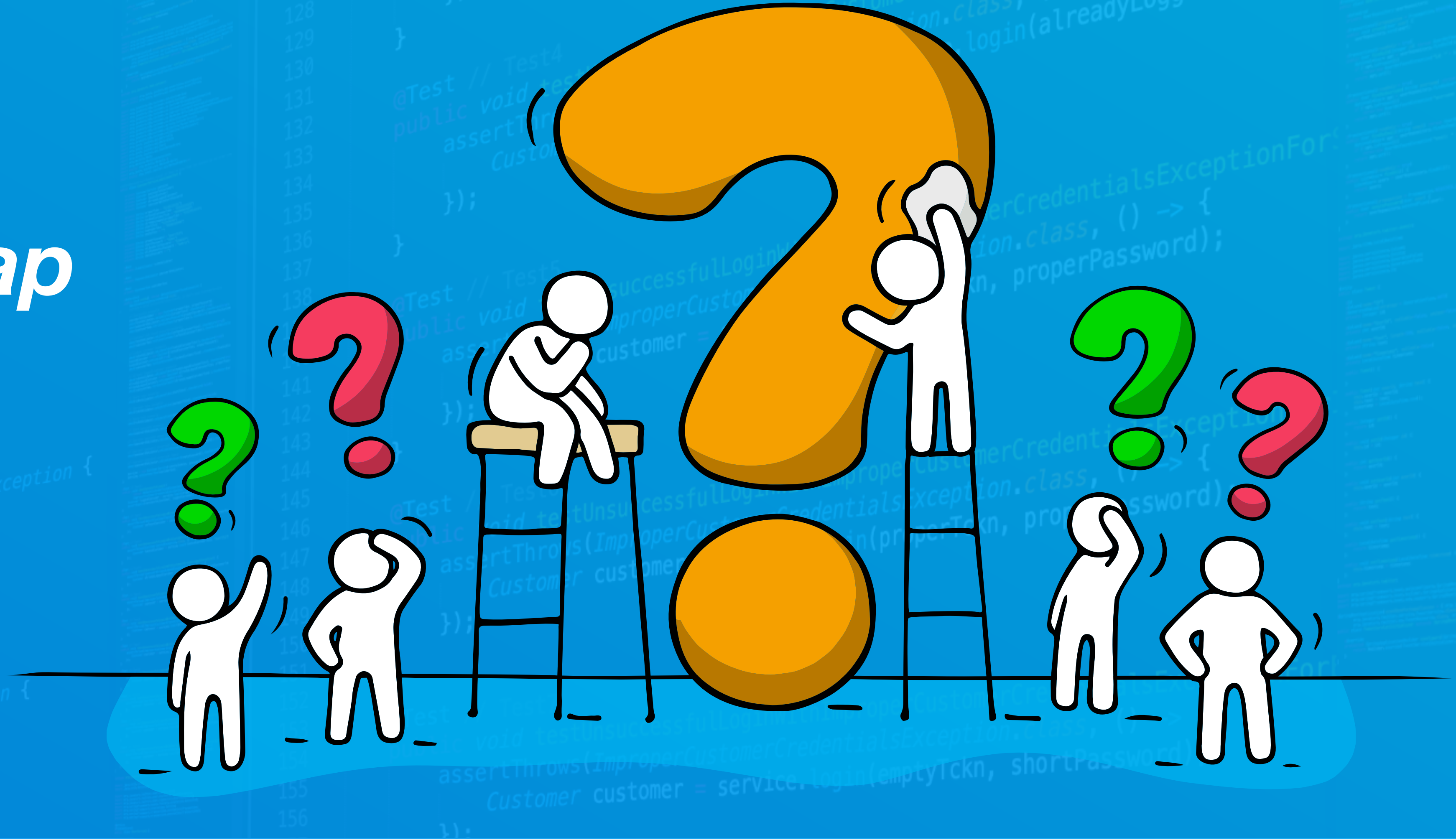
```
var var = 5;    // No problem :)
```

- Fakat **var**'ı sınıf ya da arayüz ismi olarak kullanan kodlarda bu isimler değiştirilmelidir



- **VarDemo**

# Soru ve Cevap Zamanı!





# İsimsiz Değişken

# İsimsiz Değişken - I



- Java 22 ile birlikte dile **İsimsiz Değişken (Unnamed Variable)** özelliği geldi
- <https://openjdk.org/jeps/456>
- İsimsiz değişken **\_** (alt çizgi ya da underscore) ile tanımlanır dolayısıyla tanımlanan değişkenin ismi yoktur
- **\_**'nin Java 9 ile anahtar kelime olduğunu biliyoruz

```
int _ = 5;  
String _ = "Java";
```

# İsimsiz Değişken - II



- İsimsiz değişkenler sadece yerel değişken olabilirler ve tanımlanmalıdır dolayısıyla muhakkak bir değer ataması yapılmalıdır

```
int _ = 5;  
String _ = "Java";  
long _; // Error!
```

- Üye değişkenler isimsiz olamaz çünkü onlar zaten nesneyi tasvir ederler

# İsimsiz Değişken - II



- İsimsiz değişkenlere ulaşılamaz yani isimsiz değişkenlerin kapsamı yoktur

```
String _ = "Java";  
System.out.println(_); // Error!
```

- Dolayısıyla aynı blokta birden fazla isimsiz değişken tanımlanabilir
- İsimsiz değişken **var** ile de tanımlanabilir

```
void main(){  
    int _ = 5;  
    String _ = "Java";  
    var _ = 5;  
}
```

# İsimsiz Değişken - IV



- Peki böyle saçma bir şeye dilde neden ihtiyaç duyulur?
- Bazen yapılan işin kendisi, işin ürettiği sonuçtan daha önemli olur
- Üretilen sonucu da almak istersiniz ama onu kullanmak gibi bir hedefiniz yoktur; bu durumda sonucu isimsiz değişkene atayabilirsiniz

```
int[] orderIDs = {34, 45, 23, 27, 15};  
int total = 0;  
for (int id : orderIDs) {  
    total++;  
}  
System.out.println("Total: " + total);
```



```
int[] orderIDs = {34, 45, 23, 27, 15};  
int total = 0;  
for (int _ : orderIDs) {  
    total++;  
}  
System.out.println("Total: " + total);
```

# UnnamedVariables



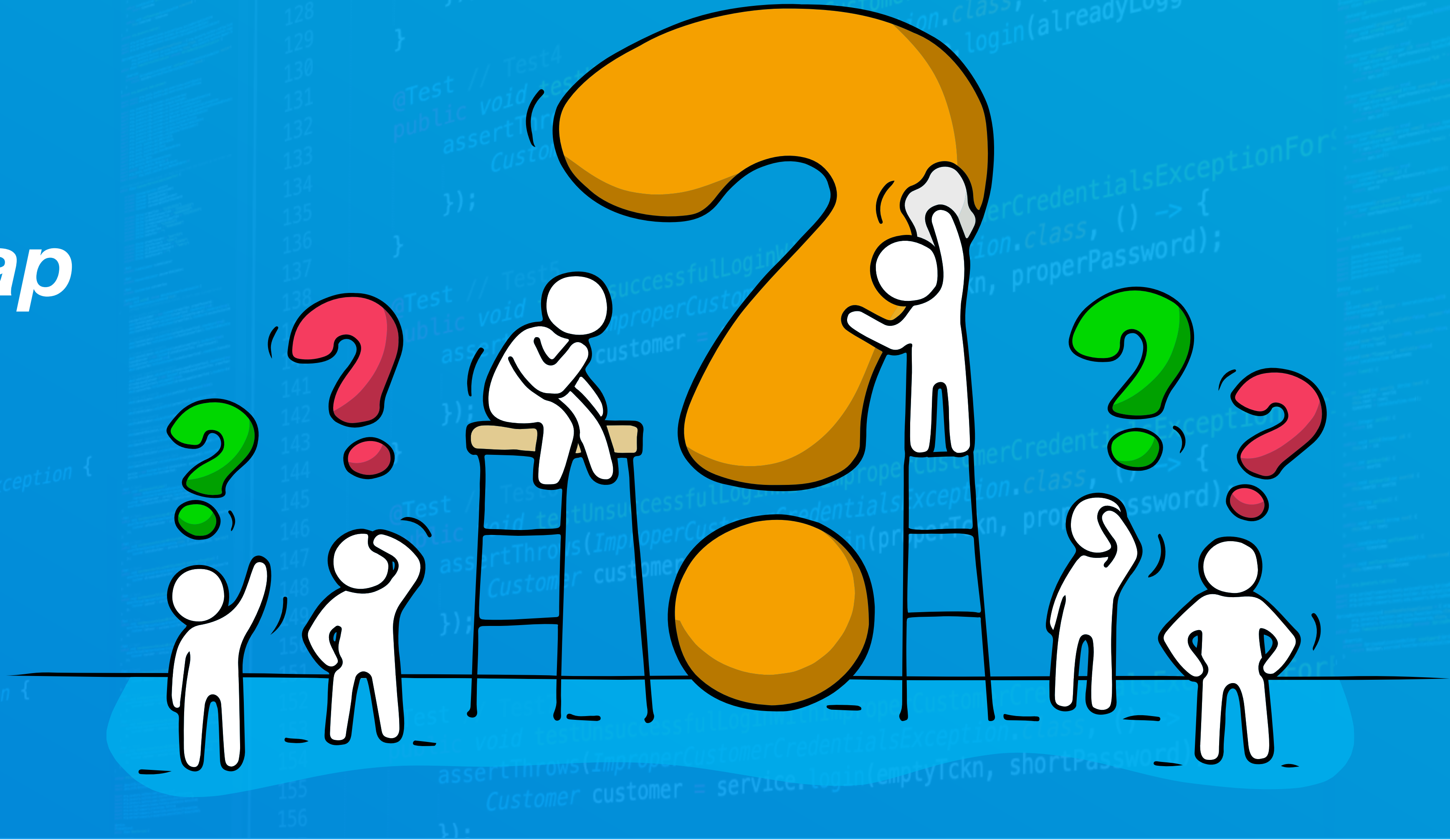
- UnnamedVariables

# İsimsiz Değişken - V



- İsimsiz değişkenler ileriki derslerde farklı konularda işimize yarayacaktır
- Dolayısıyla şimdilik bu kadar bilgi ile yetinelim

# Soru ve Cevap Zamanı!



# Bellek



# Bellek (Memory)



- Değişkenler de veriler de bellekte (memory) bulunur orada yaşar dedik
- Burada belleği programlama dilleri açısından ele alacağız, dolayısıyla fiziksel yapılarıyla ilgilenmeyeceğiz
- Fakat bellek derken genelde RAM ile CPU/GPU'nun üzerindeki cache denen ön bellek vb. alanları kastederiz
- Dolayısıyla programlama dillerindeki bellek kavramı bir çalışma zamanı (run-time) yapısıdır

# Heap ve Stack - I



- Programlama dillerinin çalışma zamanında rol oynayan en önemli iki bellek yapısı **heap** ve **stack** tir
- Dilimize stack *yığın*, *deste* ve *istif* olarak çevrilirken heap için daha çok *öbek*, *yığıt* ve *kümeleme* kelimeleri kullanılmaktadır
- İşletim sistemi bu alanları, JVM ve çalışan diğer programlara RAM, ön bellek (cache) vb. bellek yapılarından verir
- Heap ve stackin büyüklüğü JVM başlatılırken belirtilebilir

# Heap ve Stack - II



- Heap ve stack, farklı yapıdadır
- Stack, üst üste konan bellek alanlarından oluşur, bu yüzden **son giren ilk çıkar** mantığıyla (**last-in, first-out, LIFO**) çalışır
- Heap ise daha düzensiz bir yapıdadır
- Bu sebeple stack, üst üste konmuş tabaklar ya da kitaplar gibi düşünülebilirken heap daha çok bir tarlaya dizilmiş tuğla yığınları gibi düşünülebilir



# Heap

# Heap - I



- **Heap**, JVM tarafından nesne oluşturmak için kullanılan bellek alanıdır
- Dolayısıyla oluşturulan tüm nesneler, nesne değişkenleriyle birlikte heapte bulunurlar
- Heap yönetimi dolayısıyla nesnelerin oluşturulması ve zamanı geldiğinde bellekten temizlenmesi Garbage Collector/Çöp Toplayıcı (GC) tarafından halledilir
- Bellekten ve bellek sızıntısından (memory leak) bahsedildiğinde ilk akla gelendir

# Heap - II



- Eskiden bu yana **dinamik bellek (dynamic memory area)** olarak da bilinir
- Heap genelde stackten çok daha büyüktür



# Stack

# Stack - I



- **Stack**, programda o anda çalışan, aktif olan metotlar için ayrılan bellek alanlarının olduğu yerdir
  - Bu sebeple **call stack** de denir
- Stack, **last-in, first-out (LIFO)** çalışır
- Her metodun stackteki bellek alanına pencere (frame) denir ve pencereler stackte üst üste açılır

# Stack - II



- Java'da stackte daima önce `main` metodun penceresi açılır
- Sonra akışta `main`'den başlayarak çağrılan her yeni metot için stackte üstte yeni bir pencere açılarak devam eder
- Metotun çalışması bitince, akış kendisini çağırان metoda dönerken stackteki penceresi de silinir
- Bu şekilde stackteki pencereler açılma sırasının tersiyle silinir
- En son `main` metodunun bellek penceresi de silinir ve JVM durur

# Stack - III



- Stackteki bellek pencereleri, kendisi için açıldığı metodun yerel değişkenleri vs. için bellek ihtiyacını karşılar
- Stackteki bir pencerenin büyüklüğü, metodun parametreleri ve o metotta tanımlanan değişkenler vb. bellek tüketen yapıların sayısı ve büyüklüğüne bağlıdır
- Stack silinince, içindeki tüm veriler de silinir

# Stack - IV



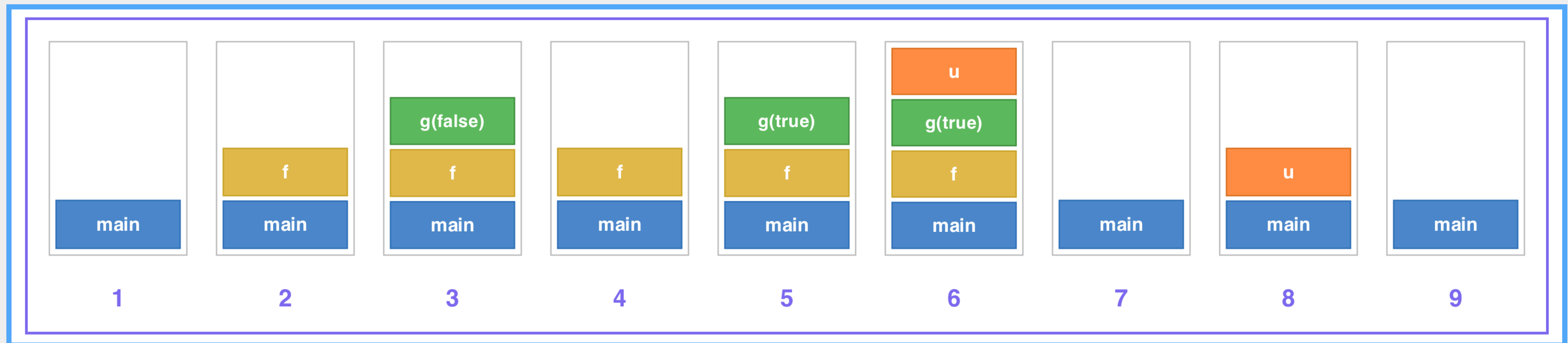
- Bazen stackte çok fazla sayıda metot penceresi açılır ve bu pencerelerin kullandıkları bellek miktarının toplamı, stack büyüklüğünü geçer, bu durumda `java.lang.StackOverflowError` hatası fırlatılır
- İleride bu konuda daha çok bilgi verilecektir

# StackDemo.java



StackDemo'nun çalışmasıyla stack belleğinin değişimi

```
... main(...){  
    //1  
    f();  
    //7  
    u();  
    //9  
}  
  
... f(){  
    //2  
    g(false);  
    //4  
    g(true);  
}  
  
... g(boolean b){  
    //3 & 5  
    if(b)  
        u();  
}  
  
... u(){  
    //6 & 8  
}
```



# StackDemo - I



- StackDemo
- Bu örnekte, programın stack belleği incelenecektir
- Programı çalıştırdığınızda, her metodunda o anki stack bilgisi yazdırılmaktadır

# StackDemo - II



- `StackDemo` örneğini debugger ile çalıştırın
- Kodda `printStackTrace()` çağrılarına *break point* koyun ve o noktalardaki stack belleğinin içindeki metod pencelerini (method frames) görün

# Yerel Değişkenler ve Stack



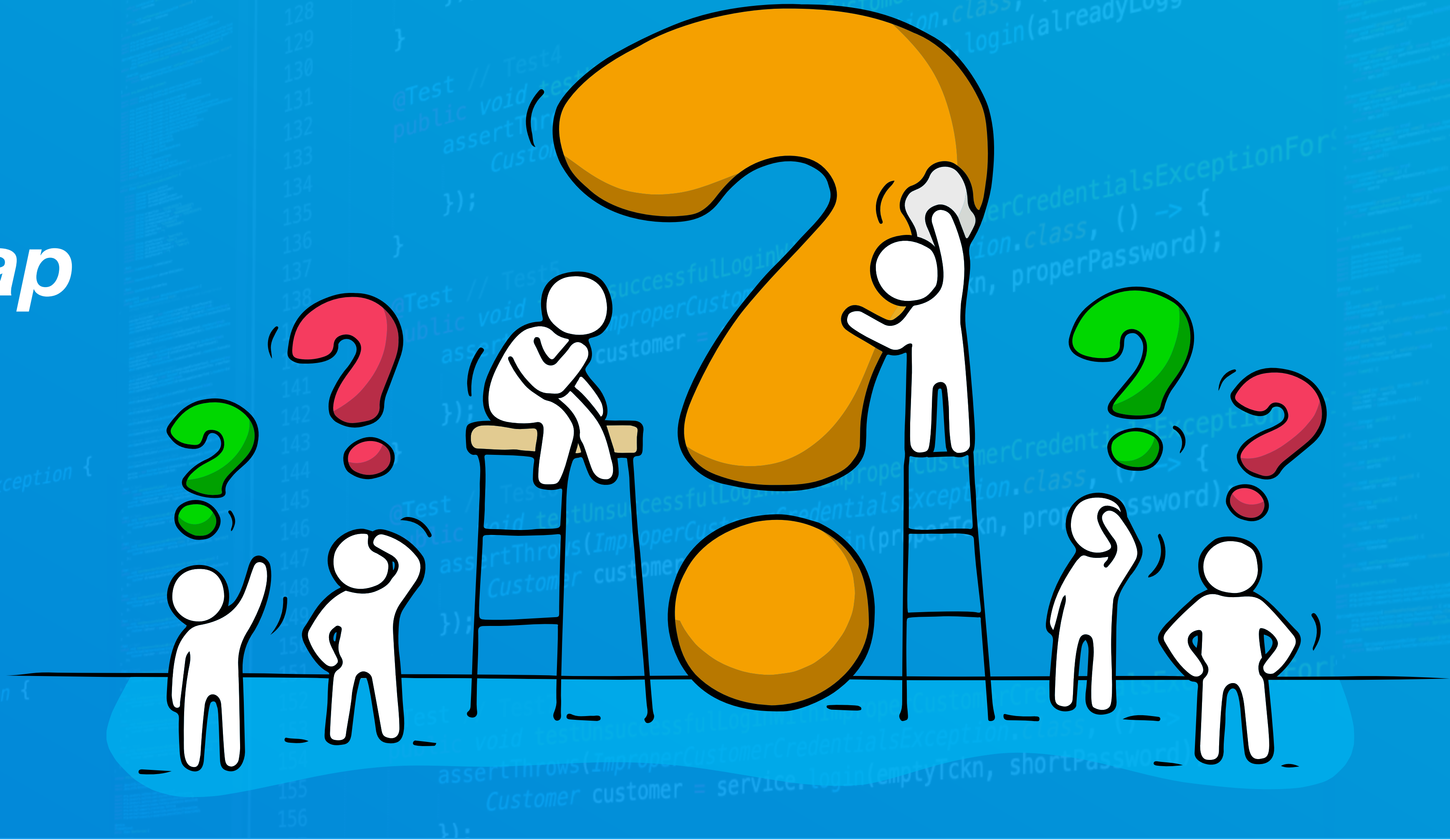
- Yerel değişkenlere aşağıdaki isimler de verilir:
- **Yığın değişkenleri (stack variables):** Çünkü nesneler heapte oluşturulurken, yerel değişkenler stackte oluşturulur
- **Otomatik değişkenler (automatic variables):** İçinde bulundukları bloğa her girişte tekrar oluşturulur, çıkınca erişilemez olurlar
- **Geçici değişkenler (temporary variables):** Ömürleri, içinde bulundukları bloğun ömrü kadardır ama nesne değişkenlerinin ömrü nesnenin ömrü ile belirlenir

# Değişkenin Tipi ve Rolü



- Yerel ya da üye değişken olmak, değişkenin tipinden bağımsız, değişkenin rolünü belirleyen bir durumdur
- İlkel bir değişken rol olarak üye bir değişken ise, içinde olduğu nesnenin yaratıldığı heapteki adres alanında bulunurken, bir metotta oluşturulan bir referans stackte bulunacaktır
- Ama stackteki bir referansın gösterdiği nesne her halükarda heapte olacaktır

# Soru ve Cevap Zamanı!





# Değişkenlerin Ömrü

# Değişkenlerin Ömrü - I



- **Değişkenin ömrü (lifetime of a variable)**, değişkenin bellekte kalmaya devam ettiği zamanı ifade eder
- Sınıf değişkenlerinin ömrü, sınıf JVM'e yüklendikten (loading a class) sınıfın JVM'den boşaltılmasına (unloading a class) kadar geçen süredir
- Tipik olarak JVM çok gerekmedikçe yüklenen bir sınıfı boşaltmaz; bu durumda sınıf değişkenlerinin ömrü program bitinceye kadardır
- Nesne değişkenlerinin ömrü, nesne yaratıldıktan nesnenin Garbage Collector (GC) tarafından toplanmasına kadar geçen süredir

# Değişkenlerin Ömrü - II



- Yerel değişkenlerinin ömrü, tanıtıldıktan içinde bulundukları metodun bitimine kadar geçen süredir
- Örneğin eğer bir yerel değişken bir metod argümanı ise, bu durumda metod penceresi stackte aktif olarak bulunduğu sürece bellekte yaşar
- Metodun çalışması bittiğinde penceresi de stackten silinir ve değişkenin ömrü biter
- Metotların iç bloklarında oluşturulan değişkenlerin ömrü, benzer şekilde içinde bulunduğu metodun stackten silinmesiyle biter



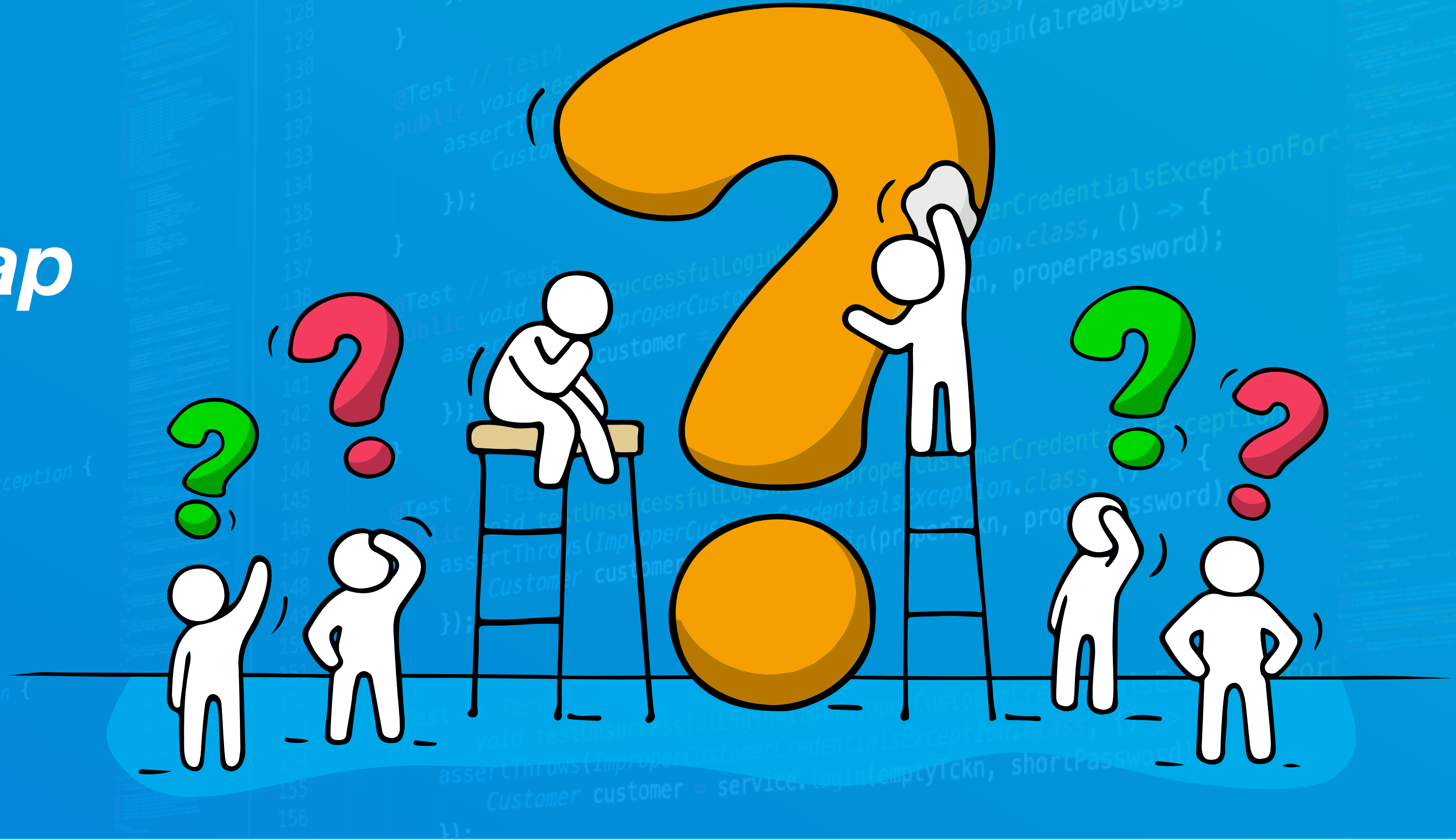
# Ödev



- Koddaki değişkenleri bulun, tiplerini ve bellekteki yerlerini (stack veya heap) belirleyin
- Değişkenlerin ömürlerini de belirleyin
- Nesnenin ömrü, onu gösteren referansın ömrüyle aynı mı? Açıklayın

```
public static void main(String[] args){  
    Car car1 = new Car();  
    car1.make = "Mercedes";  
    car1.model = "C200";  
    car1.year = "2019";  
    car1.distance = 0;  
    car1.speed = 0;  
  
    int carSpeed = car1.speed;  
  
}
```

# Soru ve Cevap Zamanı!



# Kapsam



# Kapsam - I



- Değişkenlerin erişilebildiği ya da görülebildiği alana, **kapsam (scope)** denir
- Java'da kapsam, bloklar tarafından belirlenir
- Java'da kapsam bir derleme zamanı yapısıdır (**static scoping** ya da **lexical scoping**) ve çalışma zamanında değişmez
- Bazı dillerde çalışma zamanında belirlenen dinamik kapsam (dynamic scoping) vardır

# Kapsam - II



- Üye değişkenlerin (member variables) kapsamı tüm sınıftır
- Üye değişkenin kapsamı, nerede tanıtıldığından bağımsızdır
  - Yani bir üye değişkeni sınıfın en altında da tanıtılsa kapsamı tüm sınıftır
- Yerel değişkenlerin (local variables) kapsamı, tanıtıldıkları yerden, içinde bulundukları bloğun sonuna kadardır
- Kapsama bloğun içindeki tüm alt kapsamlar da dahildir

# Car ve CarTest



- `Car` ve `CarTest`
- Bu iki sınıftaki değişkenlerin kapsamlarını inceleyin

# Kapsam - III



- Aynı kapsam içinde birden fazla aynı isimde, üye ya da yerel değişken olamaz
- Fakat aynı kapsamda aynı isimde bir üye ve bir yerel değişken olabilir
- Bu durumda yerel değişken, üye değişkeni **gölgeler (shadowing)**
- Java bu durumdaki bir değişkene erişimde, en yakında tanıtılana yani yerel değişkene erişir, üye değişkene ulaşmak için ayrı bir yöntem kullanmak gerekir

# Kapsam Dışı



- Bir değişkene kapsamının dışında erişilemez, erişmeye çalışmak derleme hatasıdır
- Bu durumda o değişkenin erişilemediği, görülemediği yani **kapsam dışı (out of scope)** olduğu söylenir

# İleri Referans (Forward Reference)



- Yerel değişkenlerde kapsam, geriye doğru işlemez
- Kapsam, kodun söz dizimine uygun olarak genişler (lexical scoping)
- Bu sebeple ileri referans (forward reference) bir derleyici hatasıdır
- Üye değişkenlerin kapsamı tüm sınıftır ama onlarda da, ilk değer atarken, ileri referans yapılamaz ama diğer durumlarda tabii olarak yapılabilir

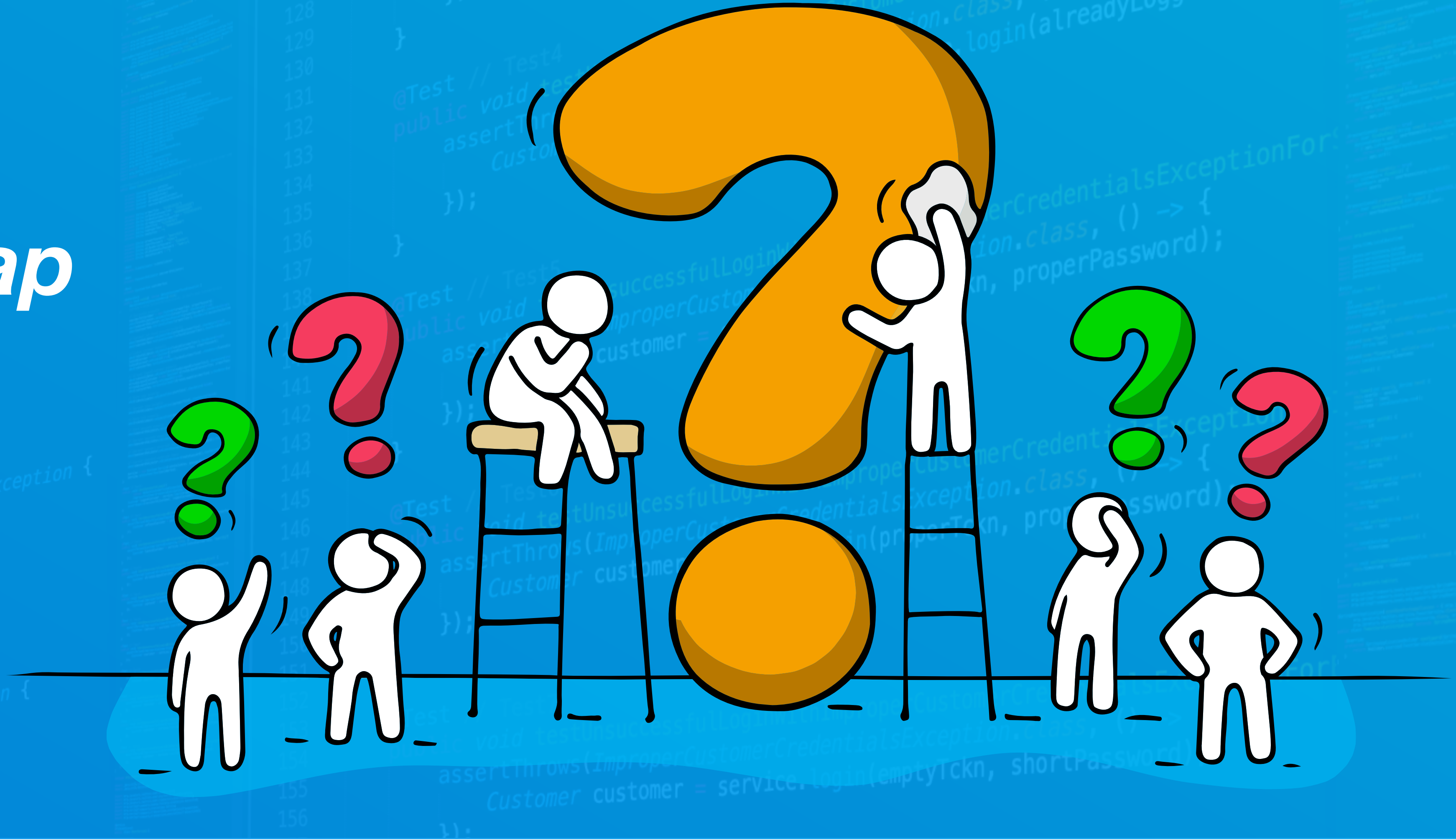
```
int i = j; // Compiler error!  
int j = 5;
```

```
class A {  
    void m() { System.out.println(j); } // Ok  
    int j = 5;  
}
```



- ScopeDemo
- Bu örnekte, değişkenlerin kapsamı ele alınacaktır
- Üye değişkenlerin tanımlandıkları yeri değiştirerek kapsamlarının tüm sınıf olduklarını gözlemleyin
- İleri referans yapılamayacağını da gözlemleyin

# Soru ve Cevap Zamanı!





# Global Değişken

# Global Değişken - I



- **Global değişken (global variable)**, programda bir yerde tanımlanıp, her yerden ulaşılabilen değişkenlere denir
- Global değişkenler, sınırsız kapsama sahiptirler
- C, C++, Python gibi diller global değişkenleri destekler

```
balance = 1000 # global

def withdraw(amount):
    global balance
    balance -= amount

def deposit(amount):
    global balance
    balance += amount
```

# Global Değişken - II



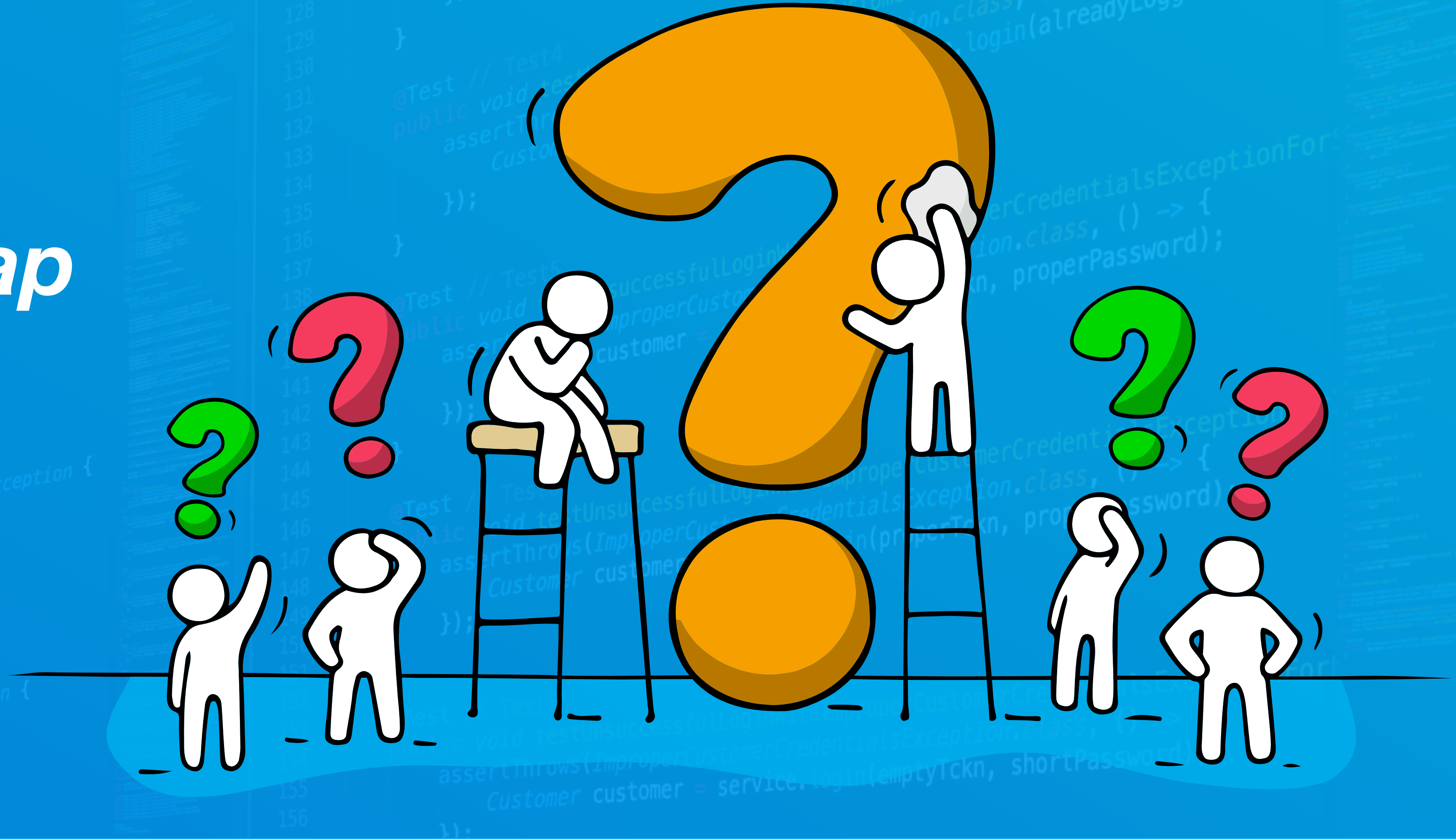
- Global değişkenin, program tasarımı açısından olumsuz olduğu iddia edilmiştir:
- [https://en.wikipedia.org/wiki/Global\\_variable](https://en.wikipedia.org/wiki/Global_variable)
- Global variable considered harmful: <https://dl.acm.org/doi/10.1145/953353.953355>

# Global Değişken - III



- **Java'da global değişken yoktur**
- Java'da tüm değişkenler (ve metotlar) ancak bir sınıfta tanımlanabilir ve ancak nesne (ya da sınıfı) üzerinden erişilebilir, dolayısıyla kısıtlı kapsamları vardır
- Ayrıca erişim belirteçleri (access modifier) yardımıyla bir sınıf ya da nesne için tanımlanan değişkene, başka sınıflardan ya da nesnelerden erişim kontrol edilebilir
- Olsa olsa ancak üye değişkenlerin *sınıfa global* olduğu söylenebilir

# Soru ve Cevap Zamanı!





# Ödev



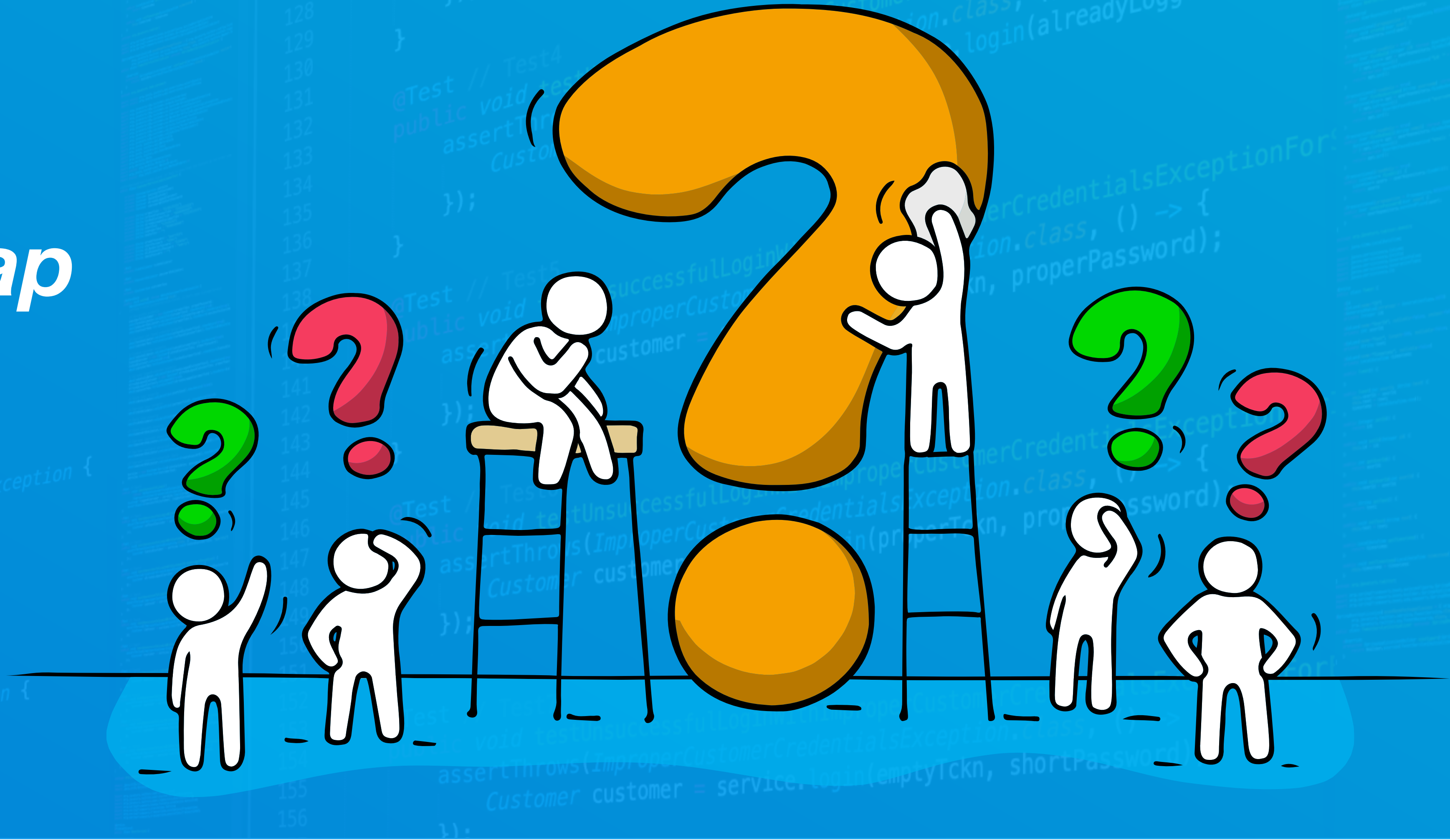
- Aşağıdaki özellik ve metotlara sahip bir `FootballPlayer` sınıfı oluşturun:
  - `no, name, inPlay, minutes, numberOfGoals`
  - `void play(int minutesToPlay),`
  - `void score()`
  - `void printInfo()`
- Sonra aşağıdaki gibi bir `FootballPlayerTest` sınıfı oluşturun ve başarıyla çalışmasını sağlayın



- Bu sınıfın testi için  
`hw.ch08.FootballPlayerTest`  
sınıfını kullanabilirsiniz

```
public class FootballPlayerTest {  
    public static void main(String[] args) {  
        FootballPlayer alex = new FootballPlayer();  
        alex.no = 10;  
        alex.name = "Alex De Souza";  
        alex.minutes = 0;  
        alex.inPlay = true;  
        alex.numberOfGoals = 0;  
  
        alex.play(90);  
        alex.score();  
        alex.score();  
  
        alex.printInfo();  
    }  
}
```

# Soru ve Cevap Zamanı!



# Değişmezlik





- Programlarda sıklıkla değişkenlerin değerlerini değiştirmemesi, daima sabit bir değere sahip olması istenir
- Değeri değişmeyen değişkenlere **değişmez (immutable)** ya da **sabite (constant)** denir
- Dilimizde *sabit* sıfattır *sabite* ise isimdir: 5 bir sabitedir yani sabit bir sayıdır
- Sabitelerin değeri sabit kalır, değişmez, yeni bir değer atanamaz

# final Anahtar Kelimesi



- **final**, Java'nın en çok sayıda farklı bağlamda kullanılan anahtar kelimesidir
- Java'da dört farklı şey **final** olabilir; burada değişkenlerin **final** olması ele alınacaktır
- **final**, bir niteleyicidir (modifier) ve nitelediği şeyin *bir anlamda* değişmemesi gerektiğini ifade eder
- Bu *bir anlam*, nitelediği şeye bağlıdır

# Sabite - I



- **final**, değişkenlerle kullanıldığında onları **sabite, değişmez (constant, immutable)** yapar
- **final**, değişken tanıılırken niteleyici olarak kullanılır
- **final** ile tanıtilan değişkenin değeri değiştirilemez, değiştirmeye çalışmak bir derleme hatasıdır

```
final int i = 5;  
i = 8; // Compiler error.
```

- **final** değişken, yerel ya da üye değişken olabilir

# Sabite - II



- **Değeri sabit, değişmeyen değişken (final/constant/immutable variable)** tamlaması garip gelebilir bu sebeple sabite (**final/constant**) demek daha anlamlı olabilir
- Java Language Specification *sabit değişken (constant variable)* tamlamasını kullanmaktadır

# Boş Sabite Tanıtımı



- Eğer değişkenin değeri çalışma zamanında belirlenecekse, değişken **boş sabite (blank final)** olarak tanımlanabilir
- Bu durumda değişken ilk değer ataması olmadan **final** olarak tanımlanır ve sonrasında bir seferlik ilk değer ataması yapılır ve değeri bir daha değiştirilemez

```
final int i = 5;  
i = 8; // Compiler error.
```

```
final int i;  
i = 8; // Not a compiler error.  
i = -3; // Compiler error.
```

- Boş sabitelerin değeri olmadığından değerlerine ulaşmaya çalışmak bir hatadır

# Sabit Referans - I



- Referansın sabite ya da değişmez olmasının anlamı, ilkel değişkenlere göre biraz farklıdır
- **final** referanslar, gösterdikleri nesneden başka bir nesneyi gösteremezler, onlara yeni bir nesne atamak derleyici hatasıdır
- Referanslar da boş değişmez olarak tanımlanabilir

```
final Car c = new Car();  
c = new Car(); // Compile-time error.
```

```
final Car c; // Blank final.  
c = new Car(); // Not a compile-time error.  
c = new Car(); // Compile-time error.
```

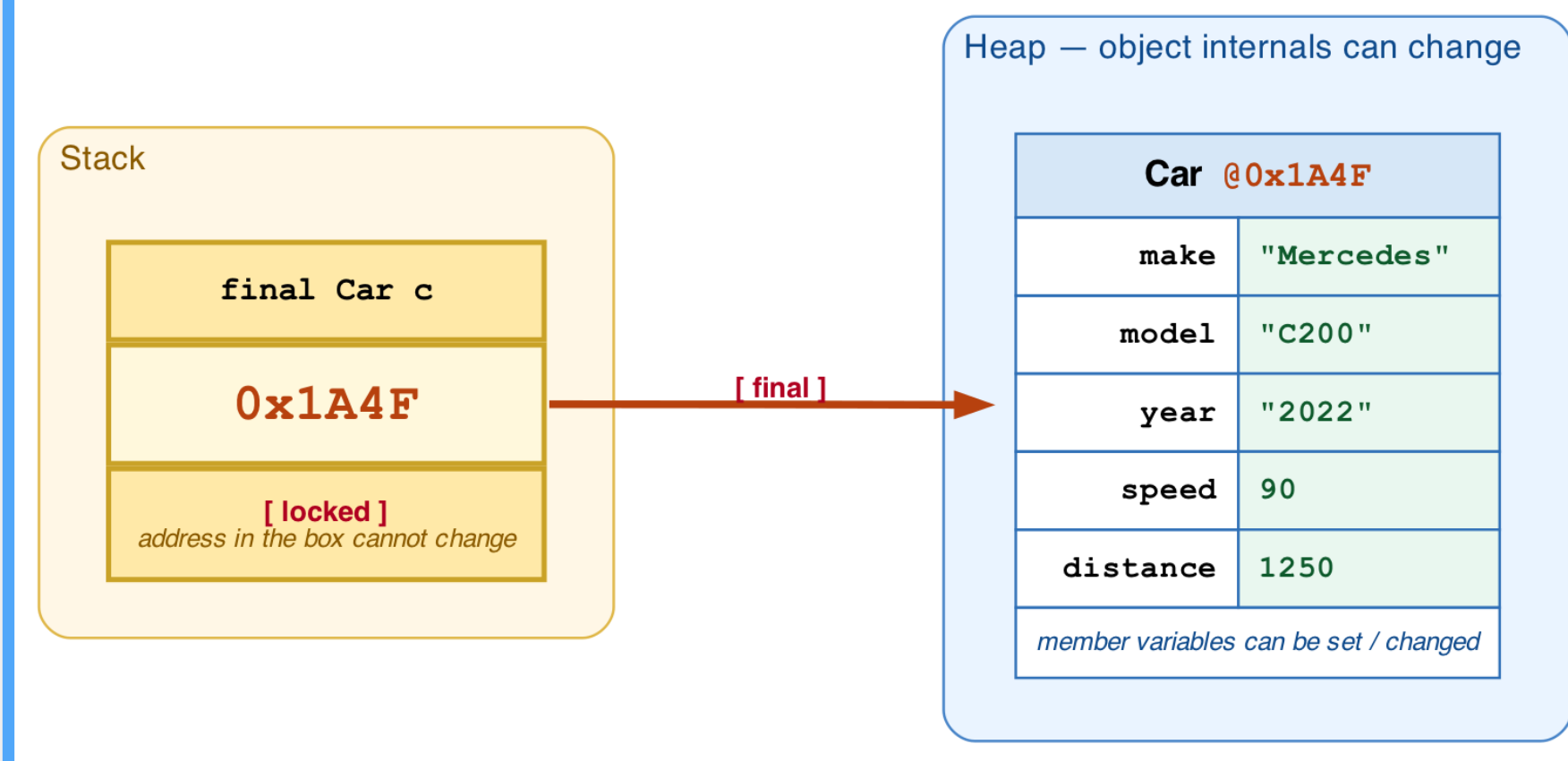
# Sabit Referans - II



- Ama **final** bir referansın gösterdiği nesnenin değişkenlerinin değerleri değiştirilebilir

```
final Car c = new Car();  
c = new Car(); // Compile-time error!  
c.make = "Mercedes"; // Not an error  
c.model = "C200"; // Not an error
```

```
final Car c = new Car();  
  
c.make = "Mercedes"; // OK  
c.model = "C200"; // OK  
c.year = "2022"; // OK  
c.speed = 90; // OK  
c.distance = 1250; // OK  
  
c = new Car(); // Compile-time error!
```



# FinalVariables



- `FinalVariables`

# const



- Java'da sabit değerli değişkenler için **final** kullanıldığından **const** anahtar kelimesinin kullanımı yoktur



Ödev



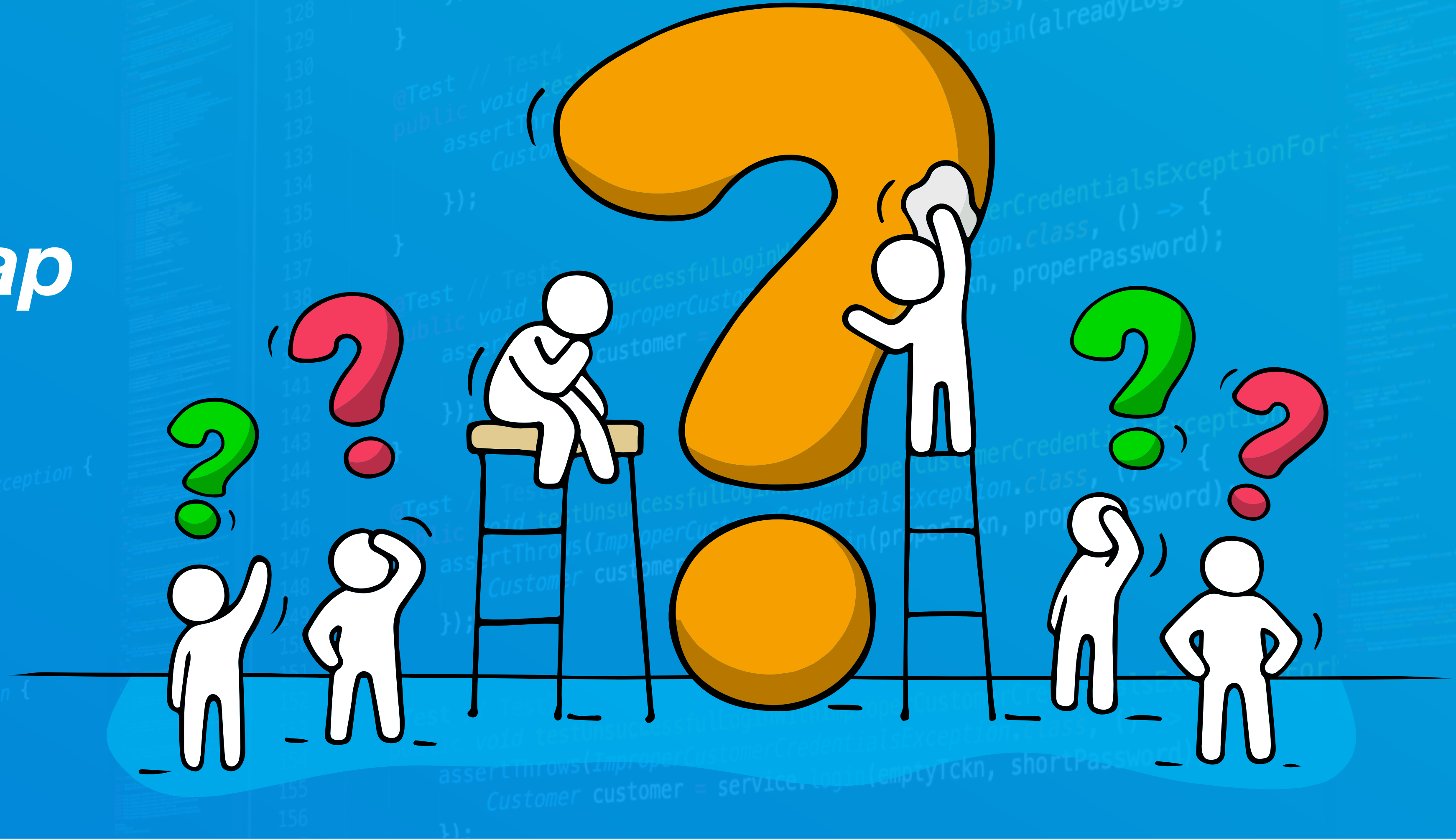
1. Yandaki iki sınıftan sizce hangisi diğerine tercih edilmelidir

a. Neden?

```
public class Point {  
    int x;  
    int y;  
    ...  
}
```

```
public class ImmutablePoint {  
    final int x;  
    final int y;  
    ...  
}
```

# Soru ve Cevap Zamanı!





# Sabit İfade ve Sabit Değişken

# Sabit İfade (Constant Expression) - I



- 3, 5.25, true, 'a' gibi ilkel değerler (primitive literal) ile “Java” gibi, değeri doğrudan iki çift kesme işareti arasında verilen `String` değerleri (`String` literal), derleme zamanı sabiteleridir, değerleri derleme zamanında bilinir
- Bu tipten değerlerle ve bazı işlemcilerden (operators) oluşan ifadelere **sabit ifade (constant expression)** denir
- Bu türden ifadelerin değeri derleme zamanında bilinir ve sonrasında da değişmez

# Sabit İfade (Constant Expression) - II



- Aşağıdakilerin hepsi ifadedir yani değeri vardır ve o değer derleme zamanında bilinebilir ve daima sabittir:
- İlkel değerler ve `String` sabiteleri ile bu tiplere yapılan cast ifadeleri
- İlkel değerler ve `+`, `-`, `*`, `/` ve `%` ile oluşturulan ifadeler
- `String` sabiteleri `+` ile ekleyen ifadeler
- Eşitlik işlemcileri `==` ve `!=`

```
5  
false  
'a'  
"Java is good!"  
2 + 7  
1 * 2 * 3 * 4 * 5 * 6  
"The integer is " + 5  
7 != 2
```

# Sabit İfade (Constant Expression) - III



- Kıyaslama işlemcileri <, <=, >, >=
- Tekli işlemciler (unary operators) +, -, ~, ve !

```
5 > 2
!true
(float) (7 * Math.PI) / 2
2.0 * Math.PI
final int i = 5;
final int j = 3;
i > j
```

# Sabit İfade (Constant Expression) - IV



- `final` olan değişkenler sabit ifadelerde kullanılabilir
- `final` olmayan bir değişken kullanılırsa ifade sabit ifade olmaktan çıkar

```
int i = 2;      // 2 is a constant expression
int j = 2 * 5; // 2 * 5 is a constant expression
int k = i * 5; // i * 5 is not a constant expression

"I love Java!"      // A constant expression
"I love Java!" + " :)" // A constant expression
String s = "I love Java!";
s + " :)" // Not a constant expression
```

# Sabit İfade (Constant Expression) - V



- `String` nesneleri ancak kurucu çağrısı yapmadan, `" "` ile oluşturulduğunda sabit ifade olabilir
- `String` dışında hiçbir karmaşık tipin nesnesi sabit ifade olamaz

```
"I love Java!" // A constant expression  
new String("I love Java!") // Not a constant expression  
  
new Car() // Not a constant expression
```

# Sabit İfade (Constant Expression) - VI



- İfadelerin değeri genel olarak çalışma zamanında belirlenmesine rağmen sabit ifadelerin değeri derleme zamanında belirlenir
- Bu sebeple örneğin metot çağırısı sabit ifade olamaz çünkü metotlar çalışma zamanı yapılarıdır

```
int i = 2;      // 2 is a constant expression
int j = 2 * 5;  // 2 * 5 is a constant expression
int k = i * 5;  // i * 5 is not a constant expression

"I love Java!"      // A constant expression
"I love Java!" + " :)" // A constant expression

String s = "I love Java!";
s + " :)" // Not a constant expression

boolean b = isEven(5); // isEven(5) is not a constant expression
```

# Sabit Değişken (Constant Variable) - I



- İlkel ya da `String` tipinden `final` bir değişkene bir sabit ifade (constant expression) atanırsa, o değişkene **sabit değişken (constant variable)** denir

```
final int i = 5;
final boolean b1 = false;
final boolean b2 = !true;
final char c = 'a';
final String s1 = "Java is good!";
final int j = 2 + 7;
final int k = 1 * 2 * 3 * 4 * 5 * 6;
final float f = (float) (7 * Math.PI) / 2;
final double d = 2.0 * Math.PI;
final String s2 = "The integer is " + 5;
final boolean b3 = 7 != 2;
```

```
final int i = 5;
final int j = 3;
final boolean b1 = i > j;

final double d1 = 2.0;
final double d2 = d1 * Math.PI;
```

# Sabit Değişken (Constant Variable) - II



- Sabit değişken, boş (blank) olarak tanıtilamaz
- Çünkü değerinin derleme zamanında belirlenebilmesi gereklidir
- Eğer bir `final` değişken boş olarak tanıtilır ve değerini tanıtımdan sonra alırsa o sadece `final` bir değişkendir, sabit değişken değildir

# Sabit Değişken (Constant Variable) - III



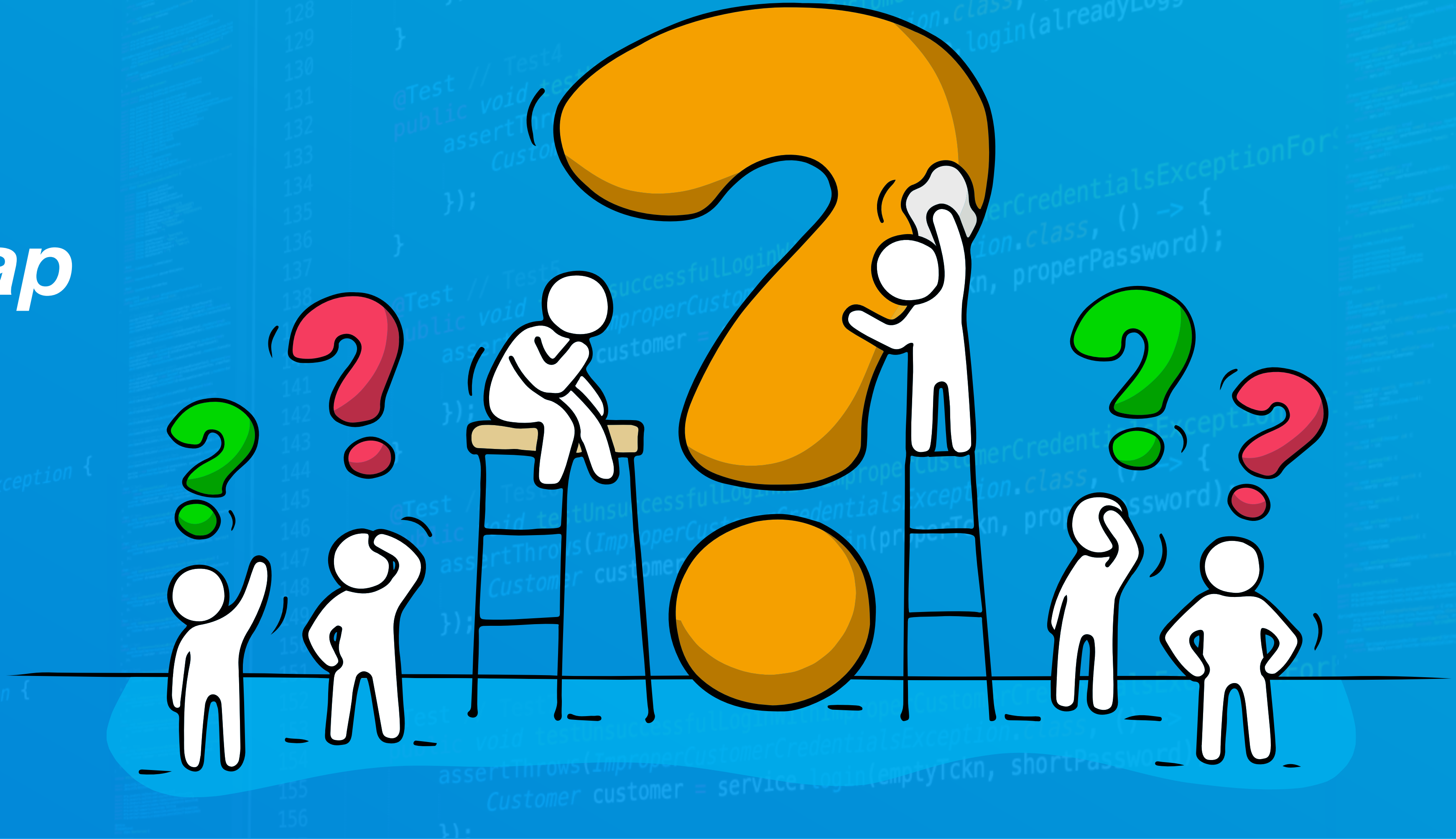
- Sabit değişkenler, yüklenme ve dilin çalışması açısından önemlidir
  - Dildeki `switch-case` gibi bazı yapılar daima sabit değişken kullanır
- Genel olarak sabit değişkenler bir kere oluşturulur, bir havuza konur ve programda her ihtiyaç duyulduğunda bu havuzdan alınarak kullanılır
- Sabit değişkenlerin değerini tanıtırken aldığı, bu değer derleme zamanında belirlenmesi gerektiği ve hiçbir zaman değişmeyeceği unutulmamalıdır

# ConstantVariables



- ConstantVariables

# Soru ve Cevap Zamanı!



# Ödevler





1. Bu kod derlenir mi? Derlenirse ne basar?
  - a. Kodu yazıp sonucu gözlemleyin
  - b. Derlenmiyorsa hatayı düzeltip çalıştırın

```
public class ScopeTest1 {  
    public static void main(String[] args) {  
        int x = 10;  
        System.out.println(x);  
        {  
            int y = 20;  
            System.out.println(x);  
            System.out.println(y);  
            x = 30;  
        }  
        System.out.println(x);  
        System.out.println(y);  
    }  
}
```



2. Bu kod derlenir mi? Derlenirse ne basar?
- a. Kodu yazıp sonucu gözlemleyin
  - b. Derlenmiyorsa hatayı düzeltip çalıştırın

```
public class LoopScope {  
    public static void main(String[] args) {  
        for (int i = 0; i < 3; i++) {  
            System.out.println(i);  
        }  
        System.out.println(i);  
  
        int j = 10;  
        for (int j = 0; j < 3; j++) {  
            System.out.println(j);  
        }  
        System.out.println(j);  
    }  
}
```



3. Bu kod derlenir mi? Derlenirse ne basar?

- a. Kodu yazıp sonucu gözlemleyin
- b. Derlenmiyorsa hatayı düzeltip çalıştırın

```
public class MethodScope {  
    static int value = 10;  
  
    public static void main(String[] args) {  
        System.out.println("Before: " + value);  
        int value = 100;  
        System.out.println("After: " + value);  
    }  
}
```



4. Bu kod derlenir mi? Derlenirse ne basar?

- Kodu yazıp sonucu gözlemleyin
- Derlenmiyorsa hatayı düzeltip çalıştırın

```
public class FinalPrimitives {  
    public static void main(String[] args) {  
        final int x = 10;  
        System.out.println(x);  
        x = 20;  
  
        final int y;  
        y = 30;  
        System.out.println(y);  
        y = 40;  
  
        int z = 50;  
        final int w = z;  
        z = 60;  
        System.out.println(w);  
        System.out.println(z);  
    }  
}
```



5. Bu kod derlenir mi? Derlenirse ne basar?
- a. Kodu yazıp sonucu gözlemleyin
  - b. Kaç tane derleme hatası var? Hepsini bulup düzeltin
  - c. Aynı kodu JShell'de yazıp deneyin, ne farkettiler?

```
public class FinalObjects {  
    public static void main(String[] args) {  
        final Car c = new Car();  
        c.make = "Toyota";  
        c.model = "Corolla";  
        c.year = "2024";  
        c.speed = 60;  
  
        c.accelerate(100);  
        c.go(50);  
        System.out.println(c.getInfo());  
  
        Car c1 = new Car();  
        c = c1;  
  
        Car c2 = null;  
        c = c2;  
    }  
}
```



6. Bu kod derlenir mi? Derlenirse ne basar?
- a. Kodu yazıp sonucu gözlemleyin
  - b. Derlenmiyorsa hatayı düzeltip çalıştırın
  - c. \* ile işaretli satırı sildiğinizde ne oluyor? Gözlemleyin ve açıklayın

```
public class ScopeTest2 {  
    static final int value = 10;  
  
    public static void main(String[] args) {  
        System.out.println(value);  
        int value = 100; // *  
        System.out.println(value);  
        value = 200;  
        System.out.println(value);  
    }  
}
```

# Bundan Sonra?



# Bundan Sonra?



- Bundan sonra işlemcileri (operators) ele alacağız

# Bölüm Sonu

## Soru ve Cevap Zamanı!

